NoSQL injections

A login without the password

`"password": "wrong"` → `{ "$ne": "" }`

A controlled comparison of five login configurations

Ercong Kee / Pheobe Neo / Kendrick Poon / Liu Qingyu

Ron et al., 2016 / controlled local reproduction

A query is a test on a record

DOCUMENT = A STORED RECORD

```
{ username: "alice", password: "…" }
```

Alice’s stored password is a non-empty string in the vulnerable lab.

EQUALITY CONDITION

```
password: "wrong"
```

Does the password equal “wrong”?

OPERATOR CONDITION

```
password: { $ne: "" }
```

Is the password different from an empty string?

Predicate = condition **$ne = not equal** **"" = empty string**

NoSQL models: document, key-value, graph, wide-column. Here: MongoDB documents.

MongoDB 7.0 $ne documentation / illustrative teaching record

Where the login goes wrong

THE ATTACKER CONTROLS THE REQUEST



Sends JSON	Builds query	Returns Alice	Remembers Alice	Allows access

The failure occurs when input changes the query condition.

Known username, no direct database access, no changes to stored records

The paper covers several attack boundaries

INPUT ENTERS…		WHAT CAN CHANGE
Query conditions	→	Operator object changes a condition Tautologies / our experiment
Query text	→	Added logic changes which records match “Union queries” / literature
JavaScript source	→	String becomes executable source JavaScript injection / comparison
A protocol command	→	A separator introduces another command Piggybacked queries / literature
A trusted browser	→	Browser sends an unwanted action Origin violation / CSRF / adjacent issue

Which controls stop the highlighted operator route?

Ron et al., 2016 / author manuscript and republication

The password value becomes a condition

1 CLIENT REQUEST

```
{"username":"alice","password":{"$ne":""}}
```

2 APPLICATION COPIES THE VALUES

```
users.findOne({
  username: body.username,
  password: body.password
})
```

3 BOTH CONDITIONS MATCH

username equals Alice
AND
password is not ""

A match becomes a session without password proof.

For this record, the stored password is non-empty. $ne: "" also matches missing fields.

An object and a string behave differently

OBJECT

```
{ "$ne": "" }
```

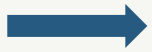


MongoDB operator

MATCH
Alice returned

STRING

```
"{\"$ne\":\"\"}"
```



Literal text

NO MATCH
Login fails

The parsed input type changes what the query means.

How we tested the controls

15 cases

Correct / wrong passwords
Operator objects
Unexpected types and fields

→

5 CONFIGURATIONS

Baseline

JavaScript disabled

Read-only DB account

String validation

Full hardened login

→

2 RUNS

Login decision
Protected access
Query count

---

150 recorded outcomes

Control probes verify settings.
Repeated runs show local repeatability.

15 cases × 5 configurations × 2 runs / results.json

Same attack, different login decision

01	02	03	04
Wrong password	**Operator object**	**Same object + fix**	**Correct password**
Rejected	**Alice session**	**Rejected**	**Alice session**

CAPTURED BYPASS

3. Application decision

```
{
  "authenticated": true,
  "account": "alice"
}
```

Alice: private synthetic project notes

CAPTURED REJECTION

3. Application decision

```
{
  "authenticated": false,
  "account": null
}
```

Protected notes: access denied

Live lab: http://127.0.0.1:4400 Offline video: 90 seconds

Actual browser captures / local synthetic lab / 1 November 2026

Which controls stopped the bypass?

Configuration	Attack result	Find queries	Valid login
Baseline	**BYPASS / 200**	1	Works / 200
JavaScript disabled	**BYPASS / 200**	1	Works / 200
Read-only DB account	**BYPASS / 200**	1	Works / 200
String validation	**BLOCKED / 400**	0	Works / 200
Full hardened login	**BLOCKED / 400**	0	Works / 200

Scripting control: this attack uses an ordinary query operator.
Read-only control: this attack needs a read, not a write.

Focal attack C04 / complete matrix in slides 17–18

A login needs separate password verification

MINIMAL TESTED CONTROL

Reject username or password if it is not a string.

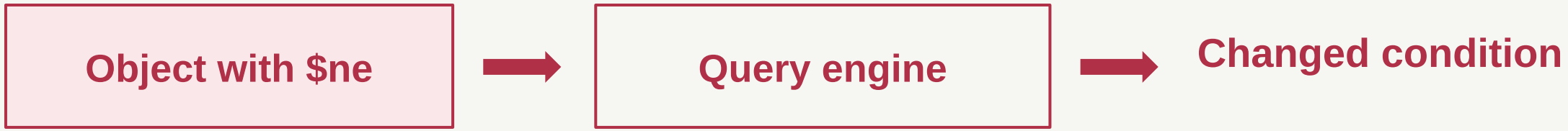
COMPLETE LOGIN DESIGN

01	02	03	04
Validate input	**Find account**	**Verify hash**	**Create session**
Exactly two fields Non-empty strings Length limits	Use validated username only	Check supplied password against stored scrypt hash	Only after verification succeeds

Local source / OWASP NoSQL and password-storage guidance

The defence depends on what reads the input

MEASURED OPERATOR INJECTION



Control: require credential strings and construct the query in application code.

LITERATURE COMPARISON



Control: avoid source construction and disable unused scripting.

Our fixed $where probe checks configuration only.

Literature comparison / MongoDB 7.0 $where documentation

What the experiment establishes

1	**The input can change the check** Operator object creates an Alice session	**200 / bypass**
2	**Validation stops this payload** Rejected before the database query	**0 finds**
3	**Correct login still works** Correct credentials retain protected access	**200 / valid**

Scope: one recorded stack, synthetic accounts and 15 request cases.

Recorded local experiment / results.json and results.csv

The application must keep control of the query

Choose the query structure.
Validate values. Verify the password.

NEXT QUESTION

How can flexible search preserve customer separation?

User-selected filter + **Mandatory tenant rule**

Proposed test: allowed filters vs a builder that always adds the tenant rule.

Recorded environment

Component	Recorded value
MongoDB	7.0.0
Node.js	v23.6.1
Express / native driver	5.1.0 / 6.20.0
Transport / parser	Loopback HTTP / JSON / express.json
Password verification	scrypt N=131072, r=8, p=1
Evidence	15 cases × 5 modes × 2 runs

Original experiment record. Fresh browser captures: 1 November 2026.

The vulnerable session decision

CLIENT VALUES ENTER THE QUERY

```
const filter = {
  username: req.body.username,
  password: req.body.password
};
const account = await users.findOne(filter);
```

```
if (account) createSession(account);
```

Condensed vulnerable excerpt. A match is incorrectly accepted as password proof.

The hardened session decision

1 Validate exactly two non-empty strings

```
if (!validContract(body)) return reject(400);
```

2 Look up by username only

```
const account = await users.findOne({ username });
```

3 Verify a salted password hash

```
const ok = await verifyPassword(password, storedHash);
```

4 Create a session after verification

```
if (account && ok) createSession(account);
```

Condensed code. Full source includes field/length checks and a dummy hash for unknown users.

Case results C01 to C08

Case	Request shape	Three toy modes	Validated / hardened
C01	Valid credentials	200	200
C02	Wrong string password	401	401
C03	Unknown username	401	401
C04	Password operator object	200	400
C05	Username operator with wrong password	401	400
C06	Operator-looking literal string	401	401
C07	Null password	401	400
C08	Array password	401	400

Toy modes = baseline, scripting disabled, read-only. Both runs agree.

200: session + access 401: failed login 400: rejected before query

Case results C09 to C15

Case	Request shape	Three toy modes	Validated / hardened
C09	Missing password	401	400
C10	Numeric password	401	400
C11	Extra field with valid credentials	200	400
C12	Top-level array	401	400
C13	Both fields contain operators	200	400
C14	Empty password	401	400
C15	Empty username	401	400

Toy modes = baseline, scripting disabled, read-only. Both runs agree.

200: session + access 401: failed login 400: rejected before query

The controls were independently checked

Control	Independent observation
Read-only identity	Authenticated as labreader / read role
Read permitted	Alice lookup succeeds
Write denied	MongoDB Unauthorized / code 13
Scripting enabled	Fixed $where expression returns Alice
Scripting disabled	Same expression fails / code 6108304

Both controls are active. Neither stops the C04 login bypass.

The fixed $where expression tests configuration, not attacker-controlled JavaScript.

Captured bypass evidence

VULNERABLE BASELINE

```
{"username":"alice","password":{"$ne":""}}
```

IDENTICAL REQUEST / ACTUAL RESPONSE REGION

HTTP 200: Session created

2. Database predicate

```
{
  "username": "alice",
  "password": {
    "$ne": ""
  }
}
```

Database query: issued

3. Application decision

```
{
  "authenticated": true,
  "account": "alice"
}
```

Alice: private synthetic project notes

Captured rejection evidence

FULL HARDENED LOGIN

```
{"username":"alice","password":{"$ne":""}}
```

IDENTICAL REQUEST / ACTUAL RESPONSE REGION

HTTP 400: Credential contract rejected

2. Database predicate

```
Rejected before filter construction
```

3. Application decision

```
{
  "authenticated": false,
  "account": null
}
```

Database query: none

Protected notes: access denied

What remains untested

ESTABLISHED

This operator bypass
Behaviour of five configurations
Rejection before a find query
Preservation of correct login

NOT ESTABLISHED

A JavaScript exploit
Real-world prevalence
All drivers or DB versions
Administrator access
Complete production security

Next experiment: permitted search without cross-tenant access.

Paper and database references

Ron, Shulman-Peleg and Puzanov (2016)
Analysis and Mitigation of NoSQL Injections
IEEE Security & Privacy 14(2), 30–39

DOI: 10.1109/MSP.2016.36

Author-uploaded manuscript and authors' InfoQ republication
Original classes: tautologies, union queries, JavaScript injection, piggybacked queries and origin violation.

MongoDB 7.0: $ne and $where documentation
Full URLs in the notes and team guide.

Defences and reproducible evidence

OWASP NoSQL Security Cheat Sheet
OWASP Password Storage Cheat Sheet

github.com/Qingyu255/CS440-Project

Evidence commit: c8252d2
Source and results are fixed at this revision.



results.json / results.csv / server.mjs / experiment.mjs
Fresh captures and team guide accompany the revised deck.